

Simple 3D “Souls-like” Prototype

This prototype includes the following core features:

- An inventory system with data persistence
- Consumable and equippable item systems
- Interaction mechanics with NPCs, objects, and items

The prototype was developed using several **software design patterns** to ensure a scalable, maintainable, and well-decoupled architecture.

ScriptableObject Data-Driven Pattern

Each item is implemented as a ScriptableObject containing metadata such as item type, a sprite for visual representation, a GameObject prefab for instantiation (object pooling could be introduced here for further optimization), item quantity, and a base abstract class that allows extension into specific item types such as consumables and equipment.

Service Locator Pattern

A Service Locator was used to manage core services, avoiding the need for multiple Singletons. This approach centralizes service access through a single Service Locator instance, improving modularity and reducing tight coupling between systems.

Strategy Pattern for Interactions

Each interactable element implements an **IInteractable** interface, allowing interaction logic to remain fully decoupled. This enables each interactable to define its own behavior while the interactor operates solely through the interface, without knowledge of the concrete implementation.

UI Architecture (MVC Pattern)

All UI components are decoupled using the MVC pattern. Views listen exclusively to events and messages to update health and mana bars, as well as to handle UI visibility (opening and closing interfaces), ensuring a clean separation of responsibilities.

The **InteractableSelectorService** is implemented as a dedicated service where all **IInteractable** objects are registered. Using parameters such as distance and dot product calculations, the system determines the most relevant interactable target and triggers **OnFocus** and **OnLostFocus** events, applying an outline effect to visually indicate which object is currently interactable **OnInteract**.

Inventory Management and Persistence

The **InventoryManagerService** orchestrates inventory logic and manages

`InventorySlot` data. Item information is injected using the `ScriptableObject` data-driven approach, allowing the UI to display item visuals, names, quantities, and descriptions. This service works in conjunction with the `InventoryDataPersistence` system, which handles saving and loading the current inventory state to and from JSON files using a generic load-save class `JsonPersistence<T>`.

Character and Player Systems

Character movement is based on Unity's third-person controller scripts, with a customized Animation Controller to support additional animations.

The `Player` is implemented as a service responsible for managing core stats such as health and mana, broadcasting updates via events to the UI. It also manages a basic weapon and armor system, handling equipped elements and their visual representation.

Conclusion

This project was a technically challenging and rewarding exercise, particularly given the time constraints. With additional time, certain areas of logic could be further refined and refactored, and potential edge-case bugs addressed. Overall, I am satisfied with the result and the architectural foundation achieved, and I appreciate the opportunity to work on this prototype.

Kind regards,
Jonathan Rincon
Senior Unity Developer